Cat Toxicity Classifier

Astrid I. Bensnes, 28.10.26

1: DESCRIBE THE PROBLEM

SCOPE

The goal of this project is to develop a tool for cat owners, where they can check whether or not an item is toxic or non-toxic to their cats. It is intended to help them identify potentially harmful foods, plants, household items and medications quickly.

Currently, in order to find this information, cat owners have to search through websites such as ASPCA Animal Poison Control database or contact a veterinarian. This can easily be time-consuming. A machine-learning-based tool like this, offers an alternative by allowing a model to learn patterns from labeled examples and make predictions for new inputs.

METRICS

To measure the model performance, several evaluation metrics were used:

- Accuracy: The amount of correctly classified items.
- Precision: The amount of items predicted as toxic, were actually labeled toxic.
- Recall: The amount of toxic items correctly identified by the model.
- F1-score: The harmonic mean of precision and recall.
- Macro F1-score: The average F1-score across both classes, giving both classes equal importance.

Recall for the toxic class was considered very important because incorrectly classifying a toxic item as non-toxic can be misleading.

Macro F1-score was used during hyperparameter tuning to evaluate performance across both classes.

AI

ChatGPT was used in this project to help understand different machine-learning algorithms, feature extraction methods and hyperparameter tuning. It was also used for coding assistance and improving wording and structure of the report. The models were trained and evaluated using the project's dataset and the final decisions were based on the experimental results.

2: DATA

2.1 Data Collection

The dataset was manually created by collecting names of foods, plants, household items, animals and medications from sources that contain information about toxicity to cats. Sources included animal welfare and poison-control resources, such as American Society for the Prevention of Cruelty to Animals (ASPCA) and Pet Poison Helpline.

Each entry contains four attributes:

- text: Name of the item.
- label: Toxic or non-toxic.
- category: Category of the item.
- source: URL of the information source.

The final dataset contains 149 entries, including 82 labeled toxic and 67 labeled non-toxic. The dataset includes the following amount in each category:

- Plant: 63.
- Food: 45.
- Household item: 26.
- Animal: 7.
- Medication: 6.
- Drink: 2.

2.2 Data Preprocessing

Before training the dataset was cleaned to ensure consistent formatting. The steps included:

1. Removing unnecessary whitespace from column names.
2. Converting item names and labels to lowercase.
3. Removing leading and trailing whitespace.
4. Removing duplicate item names.
5. Checking for missing values.

After preprocessing, the dataset contained no missing values or duplicate item names. Then, the data was divided into training and testing splits:

- 80% training data: 119 entries.
- 20% testing data: 30 entries.

The split was performed using *train_test_split* with *random_state=42* and stratification based on the labels. Stratification helped preserve nearly the same class distribution in both datasets.

2.3 Data Limitations

The dataset has a few limitations, one being that it's small with only 149 entries which means the model has limited examples from which to learn.

The category and label distribution is also uneven, with plants, foods and toxic items representing most of the dataset. When collecting data, most reliable sources focused on these, making it more difficult to even the distribution.

Another limitation is that the dataset contains only item names rather than information about them or the amount consumed. All these limitations may affect the model's ability to generalize to unfamiliar items.

3: MODELING

3.1 Model Selection

For this project I was unfamiliar with the different options for machine-learning algorithms and wasn't sure which ones would suit this project the best. In order to find something suitable for a project with a small dataset with item names, I asked AI to help me find some good alternatives. This resulted in choosing Logistic Regression and Multinomial Naïve Bayes.

Both algorithms are commonly used for text classification and can work with numerical features extracted from text. Logistic Regression was chosen because it is suitable for binary classification problems, like predicting whether an item is toxic or not. It also includes regularization, which can help reduce overfitting. Multinomial Naïve Bayes was chosen as an alternative because it is relatively simple and computationally efficient.

Although AI helped me understand the methods and suggested approaches, the models were implemented and evaluated using this project's dataset and their results were compared before deciding which model to use.

3.2 Baseline Model

Before comparing the classification algorithms, I created a baseline model using scikit-learn's *DummyClassifier* with the *most_frequent* strategy.

This approach provides a simple reference point for evaluating whether the trained models perform better than always predicting the most common class. The baseline does not learn the relationships between the item names and their labels. Since the training dataset contained more toxic than non-toxic entries, the baseline always predicted toxic. It achieved an accuracy of 56.67% on the test dataset. This result provided a minimum reference for the more advanced classification models.

3.3 Feature Extraction

Since the dataset contains item names as text, the input needed to be converted into numerical features before it could be used by the classification algorithms.

After exploring different text representation methods, I found that TF-IDF (Term Frequency-Inverse Document Frequency) is suitable for this project because it provides a relatively simple way of converting text into numerical features and works with both Logistic Regression and Multinomial Naïve Bayes.

Character-level n-grams were used with lengths between two and four characters, instead of word-level features since the dataset mainly consists of short item names. Character n-grams allow the model to identify patterns within words rather than relying on complete words.

This can help the model process unfamiliar item names that share character patterns with the training data. However, these patterns do not necessarily indicate whether an item is actually toxic.

3.4 Model Comparison

To compare the two classification algorithms, I used five-fold stratified cross-validation on the training data. Stratification was used to maintain around the same proportion of toxic and non-toxic examples in each fold. I evaluated the models using accuracy, precision, recall, F1-score and macro F1-score.

The results were:

Metric	**Logistic Regression**	**Multinomial Naïve Bayes**
Accuracy	64.71%	65.54%
Precision (toxic)	62.62%	65.25%
Recall(toxic)	89.23%	81.54%
F1-score (toxic)	73.35%	71.79%
Macro F1 score	60.13%	63.08%

Multinomial Naive Bayes achieved slightly higher accuracy and macro F1-score, but Logistic Regression achieved higher recall and F1-score for the toxic class. The Logistic Regression was chosen because identifying toxic items was an important consideration for this project. If a toxic item is classified as non-toxic, it could be very misleading.

3.5 Hyperparameter Tuning

After comparison, hyperparameter tuning was performed on Logistic Regression to see whether its performance could be improved. *GridSearchCV* with five-fold stratified cross-validation was used, because it systematically tests different combinations of hyperparameters rather than requiring each configuration to be tested manually. It included different TF-IDF analyzers and n-gram ranges to investigate how different text representations affected the results. Regularization parameter C was also tested to control overfitting and class weighting to see whether adjusting the importance of each class could improve performance.

Macro F1-score was used to select the best configuration because it gives equal importance to both classes, helping evaluate the balance between toxic and non-toxic predictions.

The selected parameters were:

Hyperparameter	**Value**
C	10
class_weight	None
analyzer	char_wb
ngram_range	(2, 4)

This selected configuration achieved a cross-validation macro F1-score of 72.33% and toxic recall of 78.46%. The higher macro F1-score suggests that the tuned configuration may provide more balanced classification performance. However, the toxic recall was lower than for the original Logistic Regression model, and the selected cross-validation score may be optimistic because it was used to choose the best hyperparameters.

3.6 Model Evaluation

The original Logistic Regression model was evaluated on the test dataset containing 30 entries. It achieved an accuracy of 66.67%, correctly classifying 20 of the 30 items. The classification report showed a recall of 76% for toxic items and 54% for non-toxic items. The confusion matrix below shows how the original Logistic Regression model classified the test entries.

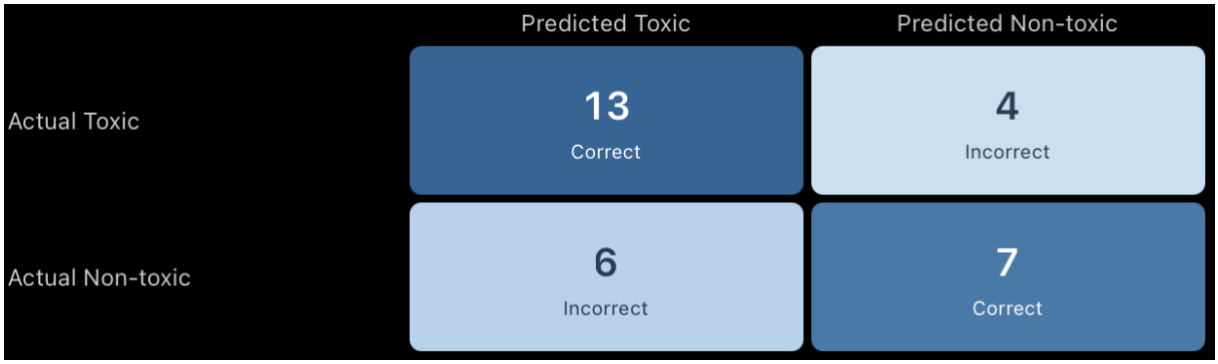


Figure 1: Confusion matrix for original Logistic Regression model

The confusion matrix shows that four toxic items were incorrectly classified as non-toxic, while six non-toxic items were incorrectly classified as toxic. These results indicate that the model learned some useful patterns from the training data, but still made several incorrect predictions. However, the false negatives are concerning because they demonstrate that the model may incorrectly predict that a toxic item is non-toxic. These results show the limitations of relying on character patterns in item names to determine toxicity.

The tuned Logistic Regression model was selected for the final web application because it achieved the highest macro F1-score during hyperparameter tuning. Although its toxic recall was lower than the original model during cross-validation, the tuned model showed more promising performance across both classes.

The tuned model was selected using cross-validation results, and its performance on the independent test dataset was not established. This means the reported results do not guarantee that it will perform equally well on new unseen items.

4: DEPLOYMENT

4.1 Web Application

The final machine-learning model was integrated into a web application developed using Streamlit. The application uses the tuned Logistic Regression model selected during hyperparameter tuning. The trained model was saved as *tuned_classifier.joblib* and is loaded into the application when needed.

When a user enters the name of an item, the input is converted to lowercase, and unnecessary whitespace is removed. The cleaned text is then passed to the saved machine-learning pipeline, which uses TF-IDF to extract features before predicting whether the item is toxic or not. The application displays the prediction directly on the website, allowing users to interact with the trained model without needing to run Python code themselves.

4.2 User Interface

The user interface was designed to be simple and easy to understand. The main purpose was to allow users to quickly enter an item and get a prediction without navigating through multiple pages.

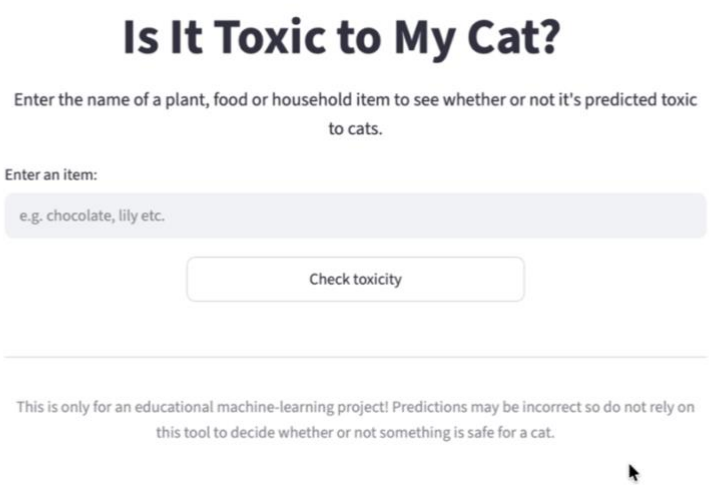


Figure 2: User input page

The interface contains a text input field where users can enter the name of an item and after submitting, the model's prediction is displayed on the page. To make results easier to distinguish, the application changes its background color depending on the prediction. A red background for toxic and a green background for non-toxic.

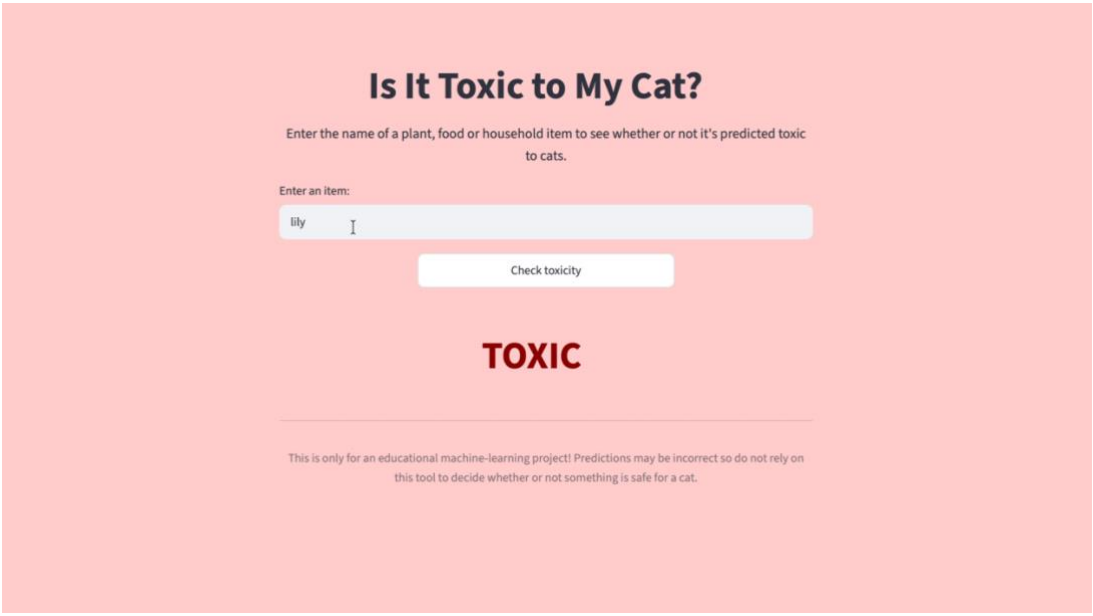


Figure 3: Toxic prediction



Figure 4: Non-toxic prediction

4.3 Future Improvements

Future improvements could include expanding the current dataset with more verified examples, particularly non-toxic items and underrepresented categories. Including common misspellings and alternative names could also help the model recognize different ways users might refer to an item.

Another improvement would be to include more information about each substance, rather than relying on its name. This could help the model learn about properties and potential effects of different substances. The model could also consider the amount of a substance, as the quantity can affect how harmful it is to a cat.

In addition, the application can be improved by handling unfamiliar inputs more appropriately instead of always returning a prediction, and by providing links to reliable sources where users can verify the results or read more about the substances.

5: REFERENCES

- ASPCA (2019, 19. November). Is it ever safe to induce vomiting? https://www.aspca.org/news/it-ever-safe-induce-vomiting
- ASPCA (n.d). People Foods to Avoid Feeding Your Pets. https://www.aspca.org/pet-care/aspca-poison-control/people-foods-avoid-feeding-your-pets
- ASPCA (n.d). Toxic and non-toxic plant list – Cats. https://www.aspca.org/pet-care/animal-poison-control/cats-plant-list
- Cat Protection (2019, 04. March). Are cleaning products safe for cats? https://www.cats.org.uk/cats-blog/are-cleaning-products-safe-for-cats
- DiPietro, R. (n.d). Pet safe plants: Flora and fauna can coexist! Pet Poison Helpline. https://www.petpoisonhelpline.com/uncategorized/pet-safe-plants-flora-and-fauna-can-coexist/
- Markdown (n.d). Markdown Reference. https://commonmark.org/help/
- Mitchell, S. C. (2023, 05. September). PetMD. What Human Food Can Cats Eat? https://www.petmd.com/cat/nutrition/what-human-foods-can-cats-eat
- Oliver, S. (2025, 14. May). PETA. What fruits and veggies can cats eat? Check out PETA's guide. https://www.peta.org/lifestyle/animal-companions/human-food-cats/
- Pet Poison Helpline (n.d). What are some foods safe to feed pets? https://www.petpoisonhelpline.com/uncategorized/what-are-some-foods-safe-to-feed-pets/
- Pet Poison Helpline (n.d). What did your pet ingest? https://www.petpoisonhelpline.com/poisons/
- Purina Editorial Team (2016, 15. October). Purina. What Human Foods Can Cats Eat? https://www.purina.co.uk/articles/cats/feeding/what-cats-eat/human-foods-for-cats
- Scikit-learn (n.d). 8.2 Feature Extraction. https://scikit-learn.org/stable/modules/feature_extraction.html
- Scikit-learn (n.d). 1. Supervised Learning. https://scikit-learn.org/stable/supervised_learning.html
- Scikit-learn (n.d). 3.2 Tuning the hyper-parameters of an estimator. https://scikit-learn.org/stable/modules/grid_search.html
- Streamlit (n.d). Get started with Streamlit. https://docs.streamlit.io/get-started